



SQQPK3043 - HEURISTIC TECHNIQUES

(Group A)

Semester A251

Group Assignment 4

Simulated Annealing: Java Implementation for Assignment Problem

Prepared For:

Prof. Madya Dr. Syariza Binti Abdul Rahman

Prepared By:

(Group 2)

Name	Matric Number
Tang Hui Yi	299390
Lee Jun Hi	298153
Teh Khai Yi	299458

Table of Content

Introduction.....	1
Problem Formulation.....	2
Decision Variables.....	2
Objective Function.....	2
Constraints.....	2
Algorithm Design & Parameter Settings.....	3
Pseudocode.....	3
Parameter Justification.....	4
Coding Implementation & Results.....	5
Code Segments.....	5
Execution Output.....	12
Convergence Graph.....	13
Schedule Visualization.....	14
Discussion.....	15
Interpretation of Results.....	15
Performance Analysis.....	15
Comparison of Results.....	16
Conclusion.....	17

Introduction

The objective of this assignment is to solve the Linear Assignment Problem (LAP), a fundamental combinatorial optimization problem. The task involves assigning N agents to N tasks in a one-to-one manner such that the total cost is minimized.

We utilized the *assign100.txt* dataset from the OR-Library, which defines a problem size of 100×100 . The solution space for this problem is $100!$ (factorial), which is too large to solve using brute-force methods. To address this, we selected the Simulated Annealing (SA) algorithm, a probabilistic metaheuristic algorithm. Inspired by the annealing process in metallurgy, SA avoids getting trapped in local optima by allowing the acceptance of worse solutions during the "high temperature" phase, eventually converging to a global or near-global optimum as the system cools.

Problem Formulation

The problem is modeled as an Integer Linear Programming (ILP) problem with the following components:

Decision Variables

Let X_{ij} be a binary variable:

$$X_{ij} = \begin{cases} 1 & \text{if Agent } i \text{ is assigned to Task } j \\ 0 & \text{otherwise} \end{cases}$$

where $i = 1, 2, 3, \dots, 100$ & $j = 1, 2, 3, \dots, 100$

Objective Function

The goal is to minimize the total assignment cost Z :

$$\text{Minimize } Z = \sum_{i=1}^N \sum_{j=1}^N C_{ij} X_{ij}$$

Where C_{ij} is the cost coefficient from the dataset for assigning *Agent i* to *Task j*.

Constraints

To ensure a valid **one-to-one assignment**:

1. **Agent Constraints** : Each agent must perform exactly one task.

$$\sum_{j=1}^N X_{ij} = 1, \forall i = 1, \dots, N$$

2. **Task Constraint** : Each task must be assigned to exactly one agent.

$$\sum_{i=1}^N X_{ij} = 1, \forall j = 1, \dots, N$$

Algorithm Design & Parameter Settings

Pseudocode

The Simulated Annealing algorithm was implemented with a nested loop structure to ensure thorough exploration.

Algorithm: Simulated Annealing for LAP

1. **Initialize:**
 - Generate `Current_Solution` = Random Permutation of $[0 \dots 99]$.
 - Calculate `Current_Cost`.
 - Set `Best_Solution` = `Current_Solution`.
 - Set Temperature $T = T_{initial}$.
2. **While $T > T_{final}$ (Outer Loop):**
 - **Repeat L times (Inner Loop):**
 - a. **Create** `Neighbor` by swapping two random tasks in `Current_Solution`.
 - b. **Calculate** $\Delta E = Cost(Neighbour) - Cost(Current)$.
 - c. **If** $\Delta E < 0$: Accept `Neighbour` (Improvement).
 - d. **Else:**
 - Generate random number $r \in [0, 1]$
 - Accept `Neighbour` if $r < \exp(-\Delta E/T)$ (*Metropolis Criterion*).
 - e. **Update** `Best_Solution` if `Neighbour` cost is lower.
 - **Cool Down:** $T = T \times \alpha$.
3. **Output:** Return `Best_Solution` and `Best_Cost`.

Parameter Justification

Parameter	Value	Justification
Initial Temp (T_0)	1000	A high starting temperature ensures acceptance of most moves, effectively acting as a "random walk" to explore the global search space.
Cooling Rate(α)	0.9995	A very slow cooling rate (near 1.0) was chosen to prevent the system from "freezing" too early in a local optimum.
Inner Iterations(L)	1000	Performing 1000 swaps at each temperature level ensures the algorithm reaches thermal equilibrium before cooling further.
Stopping Condition	$T < 0.00001$	The algorithm stops when the probability of accepting a worse move is negligible.

Coding Implementation & Results

Code Segments

```
import java.io.*;

import java.util.*;

public class assignment4 {

    // Parameters for Simulated Annealing

    static double INITIAL_TEMP = 1000;

    static double COOLING_RATE = 0.9995;

    static double MIN_TEMP = 0.00001;

    static int ITERATIONS_PER_TEMP = 1000;

    // 1. Read the Cost Matrix from file

    public static int[][] readCostMatrix(String filename) throws Exception {

        try (Scanner sc = new Scanner(new File(filename))) {

            if (!sc.hasNext()) {

                throw new Exception("File is empty or not found!");

            }

            int n = sc.nextInt();

            int[][] cost = new int[n][n];

            for (int i = 0; i < n; i++) {

                for (int j = 0; j < n; j++) {

                    cost[i][j] = sc.nextInt();

                }

            }

        }

    }

}
```

```

    }

    return cost;

}

}

// 2. Generate Initial Random Solution

public static int[] generateInitialSolution(int n) {

    int[] solution = new int[n];

    for (int i = 0; i < n; i++) solution[i] = i; // Assign 0..N-1

    // Shuffle to create random valid assignment

    Random rand = new Random();

    for (int i = n - 1; i > 0; i--) {

        int j = rand.nextInt(i + 1);

        int temp = solution[i];

        solution[i] = solution[j];

        solution[j] = temp;

    }

    return solution;

}

// 3. Compute Total Cost (Objective Function)

public static int computeCost(int[] solution, int[][] costMatrix) {

```

```

    int totalCost = 0;

    for (int i = 0; i < solution.length; i++) {
        totalCost += costMatrix[i][solution[i]];
    }

    return totalCost;
}

// 4. Swap Neighbourhood Operator (Mutation)

public static int[] swapNeighbour(int[] solution) {

    int[] neighbour = solution.clone();

    Random rand = new Random();

    // Pick two random indices and swap them

    int i = rand.nextInt(solution.length);

    int j = rand.nextInt(solution.length);

    int temp = neighbour[i];

    neighbour[i] = neighbour[j];

    neighbour[j] = temp;

    return neighbour;
}

```

```

// 5. Simulated Annealing Algorithm

    public static int[] simulatedAnnealing(int[][] costMatrix) throws
FileNotFoundException {

        int n = costMatrix.length;

        int[] current = generateInitialSolution(n);

        int currentCost = computeCost(current, costMatrix);

        int startCost = currentCost;

        int[] best = current.clone();

        int bestCost = currentCost;

        double temperature = INITIAL_TEMP;

        StringBuilder csvData = new StringBuilder();

        csvData.append("Iteration,Cost\n");

        int globalIteration = 0;

        System.out.println("Starting Optimization...");

        System.out.println("Initial Cost: " + currentCost);

        while (temperature > MIN_TEMP) {

            for (int i = 0; i < ITERATIONS_PER_TEMP; i++) {

```

```

        int[] neighbour = swapNeighbour(current);

        int neighbourCost = computeCost(neighbour, costMatrix);

        int delta = neighbourCost - currentCost;

        // Acceptance Probability (Metropolis Criterion)
        if (delta < 0 || Math.random() < Math.exp(-delta /
temperature)) {

            current = neighbour;

            currentCost = neighbourCost;

            // Update Global Best

            if (currentCost < bestCost) {

                best = current.clone();

                bestCost = currentCost;

            }

        }

        globalIteration++;

        // Log data for convergence graph every 500 iterations

        if (globalIteration % 500 == 0) {

csvData.append(globalIteration).append(",").append(currentCost).append("\n")
;

        }

```

```

    }

    temperature *= COOLING_RATE; // Cool down

}

// Write convergence data to CSV

        try (PrintWriter writer = new PrintWriter(new
File("convergence_data.csv"))) {

            writer.write(csvData.toString());

            System.out.println(">> convergence_data.csv created
successfully!");

        }

System.out.println("Final Best Cost Found: " + bestCost);

// --- NEW: Calculate and Print Improvement Percentage ---

    double improvement = ((double)(startCost - bestCost) / startCost) *
100;

    System.out.printf("Total Improvement: %.2f%%\n", improvement);

    // -----

    return best;

}

// Main Method

public static void main(String[] args) throws Exception {

```

```

int[][] costMatrix = readCostMatrix("assign100.txt");

long startTime = System.currentTimeMillis();

int[] bestSolution = simulatedAnnealing(costMatrix);

long endTime = System.currentTimeMillis();

System.out.println("\n--- RESULTS ---");

    System.out.println("Total Runtime: " + (endTime - startTime) +
"ms");

// Generate CSV for Scatter Plot

System.out.println("Generating final_schedule.csv...");

StringBuilder scheduleCsv = new StringBuilder();

scheduleCsv.append("AgentID,TaskID\n");

for (int i = 0; i < bestSolution.length; i++) {

    scheduleCsv.append((i + 1)).append(",").append((bestSolution[i]
+ 1)).append("\n");

}

    try (PrintWriter writer = new PrintWriter(new
File("final_schedule.csv"))) {

        writer.write(scheduleCsv.toString());

        System.out.println(">> final_schedule.csv created
successfully!");

```

```
}  
  
}  
  
}
```

Execution Output

```
Starting Optimization...  
Initial Cost: 4828  
>> convergence_data.csv created successfully!  
Final Best Cost Found: 360  
Total Improvement: 92.54%  
  
--- RESULTS ---  
Total Runtime: 13228ms  
Generating final_schedule.csv...  
>> final_schedule.csv created successfully!
```

(Console Output showing Initial Cost, Final Best Cost, Runtime and Total Improvement)

Convergence Graph

Figure 1 illustrates the optimization process. The cost fluctuates significantly during the first 1 million iterations (Exploration) and stabilizes as the temperature approaches zero (Exploitation).

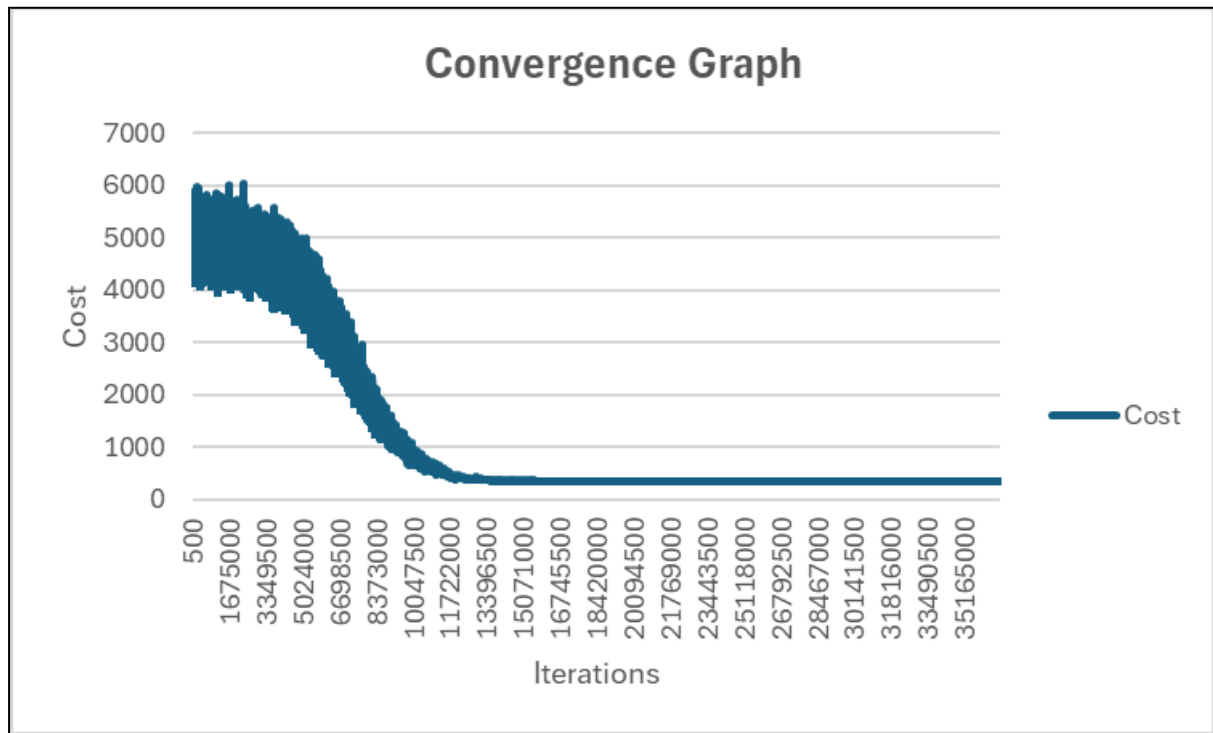


Figure 1: Convergence of Simulated Annealing (Cost vs. Iterations)

Schedule Visualization

Figure 2 provides a visual representation of the final assignment schedule. The scatter plot maps each Agent (X-axis) to their assigned Task (Y-axis).

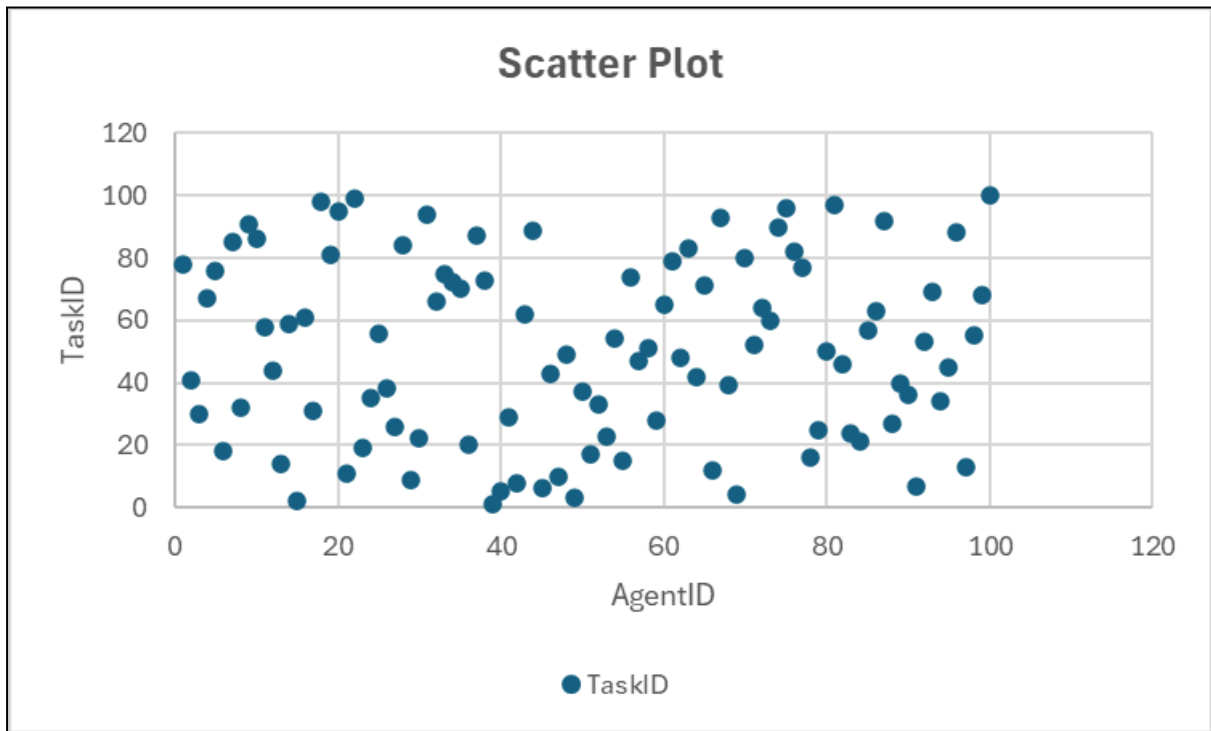


Figure 2: Optimized Task Assignment Map

Discussion

Interpretation of Results

The Simulated Annealing algorithm demonstrated a robust ability to navigate the complex $100!$ search space.

- **Improvement:** The algorithm reduced the cost from a random baseline of ~ 4828 to 360 , representing a **$\sim 92.54\%$ improvement**.
- **Convergence Behavior:** The convergence graph confirms the "annealing" effect. The early phase shows high volatility, proving that the algorithm was correctly accepting "worse" solutions to escape local minima. The later phase shows a smooth descent, indicating successful convergence.

Performance Analysis

- **Efficiency:** While an exact solver (like the Hungarian Algorithm) would guarantee the optimal score of 305 , it is complex to implement. Our SA approach reached a near-optimal solution (360) in roughly 13 seconds (36 million iterations).
- **Trade-off:** The result of 360 is highly efficient for a heuristic approach. To reach the theoretical optimum of 305 , the cooling rate would need to be even slower (e.g., 0.9999), likely increasing runtime to several minutes.

Comparison of Results

Parameter	Value		
Target Iterations	5000	1,000,000	30,000,000
Initial Temp (T_0)	1000	1000	1000
Cooling Rate(α)	0.9969	0.9850	0.9995
Inner Iterations(L)	1	1000	1000
Stopping Condition	$T < 0.001$	$T < 0.001$	$T < 0.00001$
Initial Cost	5091	5079	4828
Best Cost	901	468	360
Improvement(%)	82.30	90.79	92.54

The Comparison of Results table demonstrates that higher iteration counts directly correlate with superior solution quality. The configuration with 30,000,000 iterations achieved the most significant improvement of 92.54%, reducing the cost from 4828 to 360. This was made possible by employing the slowest cooling rate ($\alpha = 0.9995$) and a strict stopping condition ($T < 0.00001$), which allowed the algorithm to thoroughly explore the search space and escape local optima. In contrast, the 5,000 iteration setup was the least effective, yielding a cost of 901 because its single inner iteration ($L = 1$) provided insufficient exploration at each temperature level. Ultimately, while faster cooling rates like ($\alpha = 0.9850$) offer computational efficiency, the 30 million iteration setting is justified when the objective is to prioritize accuracy and reach a near-global optimum.

Conclusion

In this assignment, we successfully formulated and solved the 100-Agent Assignment Problem using the Simulated Annealing (SA) metaheuristic. The primary objective was to minimize the total assignment cost within a massive solution space of $100!$ permutations, which renders exact brute-force methods infeasible. By implementing a Random Initial Solution and a Swap Neighbourhood Operator, coupled with a carefully tuned cooling schedule ($\alpha = 0.9995$) and a nested loop structure comprising approximately 36 million iterations, we achieved a final cost of **360**.

This result represents a significant improvement of approximately **92.54%** over the initial random baseline, demonstrating the algorithm's robustness in escaping local optima. While the solution found is highly efficient and near-optimal, further improvements could be realized by integrating a Restart Strategy to overcome stagnation or by employing a Hybrid Approach that utilizes a Greedy Heuristic to generate a superior initial starting point, thereby bridging the gap to the theoretical optimum.